

Shortest Unique Prefix of Every String — Trie

Pattern: Trie + Prefix Frequency

Main Idea: Build a Trie and store how many words pass through every prefix node. For each word, the first prefix whose count is 1 is its shortest unique prefix.

1. Problem Statement

Given an array of strings `arr[]`, find the shortest prefix of every string that uniquely identifies it among all strings in the array.

A prefix is unique if no other string starts with that prefix.

Example 1

Input:

```
["zebra", "dog", "duck", "dove"]
```

Output:

```
["z", "dog", "du", "dov"]
```

2. Understanding the Example

Word: **zebra**

Try the first prefix:

```
"z"
```

Only **zebra** starts with "z".

Therefore:

```
Answer = "z"
```

Word: **dog**

Try prefixes one by one.

Prefix: "**d**"

Words starting with "**d**":

dog
duck
dove

Count:

3

So "**d**" is not unique.

Prefix: "**do**"

Words starting with "**do**":

dog
dove

Count:

2

So "**do**" is not unique.

Prefix: "**dog**"

Words starting with "**dog**":

dog

Count:

1

Therefore:

Answer = "dog"

Word: duck

Prefix: "d"

Words:

dog
duck
dove

Count:

3

Not unique.

Prefix: "du"

Words:

duck

Count:

1

Therefore:

Answer = "du"

Word: **dove**

Prefix: **"d"**

Count:

3

Not unique.

Prefix: **"do"**

Words:

dog
dove

Count:

2

Not unique.

Prefix: **"dov"**

Words:

dove

Count:

1

Therefore:

Answer = "dov"

Final Output

```
["z", "dog", "du", "dov"]
```

3. Second Example

Input:

```
["geeksgeeks", "geeksquiz", "geeksforgeeks"]
```

Output:

```
["geeksg", "geeksq", "geeksf"]
```

All three words share:

```
g
ge
gee
geek
geeks
```

Therefore:

```
count("g")      = 3
count("ge")     = 3
count("gee")    = 3
count("geek")   = 3
count("geeks")  = 3
```

After "geeks" the words split:

```
geeksgeeks → geeksg
geeksquiz  → geeksq
geeksforgeeks → geeksf
```

Each of these prefixes has count 1.

Therefore:

```
geeksgeeks    → geeksg  
geeksquiz     → geeksq  
geeksforgeeks → geeksf
```

4. The Core Question

For every word, we need to find:

The smallest prefix that no other word shares.

For example:

```
dog  
duck  
dove
```

For **dog**:

"d"

is shared by:

```
dog  
duck  
dove
```

Therefore:

"d" → not unique

Next:

"do"

is shared by:

```
dog
dove
```

Therefore:

```
"do" → not unique
```

Next:

```
"dog"
```

is used only by:

```
dog
```

Therefore:

```
"dog" → unique
```

The key observation is:

The answer is the first prefix whose frequency is exactly 1.

5. Brute Force Approach

Before using a Trie, solve the problem directly.

For every word:

```
word = "duck"
```

Try every prefix:

```
"d"
"du"
```

```
"duc"  
"duck"
```

For every prefix:

```
Count how many words start with that prefix.
```

The first prefix whose count is 1 is the answer.

6. Brute Force Algorithm

```
For every word:  
  
    For prefix length from 1 to length of word:  
  
        prefix = word[:prefix_length]  
  
        count how many words start with prefix  
  
        if count == 1:  
  
            answer = prefix  
  
            break
```

7. Brute Force Code

```
def shortest_unique_prefixes(arr):  
  
    result = []  
  
    for word in arr:  
  
        for i in range(1, len(word) + 1):  
  
            prefix = word[:i]  
  
            count = 0  
  
            for other in arr:  
  
                if other.startswith(prefix):
```



```
        count += 1

    if count == 1:

        result.append(prefix)
        break

    return result
```

8. Brute Force Dry Run

Input:

```
arr = ["zebra", "dog", "duck", "dove"]
```

Processing **zebra**

Try:

```
"z"
```

Check every word:

```
zebra → starts with "z" → yes
dog   → starts with "z" → no
duck  → starts with "z" → no
dove  → starts with "z" → no
```

Count:

```
1
```

Therefore:

```
answer = "z"
```

Processing **dog**

Try:

"d"

Check:

dog → yes
duck → yes
dove → yes

Count:

3

Not unique.

Try:

"do"

Check:

dog → yes
duck → no
dove → yes

Count:

2

Not unique.

Try:

"dog"

Check:

```
dog   → yes  
duck  → no  
dove  → no
```

Count:

```
1
```

Therefore:

```
answer = "dog"
```

9. Brute Force Complexity

Let:

```
N = number of words  
L = maximum length of a word
```

For every word:

```
N
```

we try up to:

```
L prefixes
```

For every prefix, we scan:

```
N words
```

Checking whether a word starts with a prefix can itself take up to:

```
O(L)
```

Therefore the worst-case complexity is approximately:

$$O(N^2 \times L^2)$$

The important problem is the repeated work:

```
For every word
  For every prefix
    Scan all words again
```

10. Where Is the Repeated Work?

Consider:

```
dog
duck
dove
```

For every word, we repeatedly ask:

```
How many words start with "d"?
```

Then:

```
How many words start with "do"?
```

Instead of repeatedly scanning all strings, we can calculate and store this information once.

This is exactly what a Trie does.

11. Why Trie?

A Trie stores strings character by character.

For:

dog

The path is:

```
root
|
d
|
o
|
g
```

For:

```
dog
duck
dove
```

The common prefixes are represented by common Trie nodes.

Conceptually:

```
root
|
d
├── o
│   ├── g
│   └── v
│       └── e
└── u
    ├── c
    └── k
```

Every Trie node represents a prefix.

For example:

```
Node after "d"    → prefix "d"
Node after "do"   → prefix "do"
Node after "dog"  → prefix "dog"
```

```
Node after "du" → prefix "du"  
Node after "dov" → prefix "dov"
```

Therefore:

If every Trie node stores how many words pass through it, we can immediately know whether that prefix is unique.

12. Basic Trie Structure

A basic Trie node usually contains:

```
children  
is_end
```

Example:

```
class TrieNode:  
  
    def __init__(self):  
  
        self.children = [None] * 26  
  
        self.is_end = False
```

For this problem, we need a different piece of information:

```
count
```

So our node becomes:

```
class TrieNode:  
  
    def __init__(self):  
  
        self.children = [None] * 26  
  
        self.count = 0
```

13. What Does **count** Mean?

Suppose the words are:

```
dog
duck
dove
```

The relevant prefix counts are:

Prefix	Count
d	3
do	2
dog	1
du	1
duc	1
duck	1
dov	1
dove	1

Why?

Prefix **"d"**

All three words start with **"d"**:

```
dog
duck
dove
```

Therefore:

```
count("d") = 3
```

Prefix **"do"**

Two words start with **"do"**:

```
dog
```

dove

Therefore:

```
count("do") = 2
```

Prefix "dog"

Only one word starts with "dog":

dog

Therefore:

```
count("dog") = 1
```

Prefix "du"

Only one word starts with "du":

duck

Therefore:

```
count("du") = 1
```

Prefix "dov"

Only one word starts with "dov":

dove

Therefore:


```
count("dov") = 1
```

14. The Main Trie Observation

For every word:

```
Start from the root.  
  
Follow its characters.  
  
At every Trie node:  
  
    Check the count.  
  
The first node where:  
  
    count == 1  
  
gives the shortest unique prefix.
```

The formula is:

```
First prefix where count == 1
```

15. Building the Trie

Suppose we insert:

```
dog  
duck  
dove
```

Insert **dog**

Start:

```
current = root
```

Character:

```
d
```

Create node for **d**.

Increment its count:

```
d → count = 1
```

Next character:

```
o
```

Create node for **o**.

Increment:

```
do → count = 1
```

Next character:

```
g
```

Create node for **g**.

Increment:

```
dog → count = 1
```

Current Trie information:

```
d   → 1  
do  → 1
```

```
dog → 1
```

Insert **duck**

Start from root.

Character:

```
d
```

The **d** node already exists.

Increment:

```
d → count = 2
```

Next:

```
u
```

Create a new node.

Increment:

```
du → count = 1
```

Next:

```
c
```

Create a new node.

Increment:

```
duc → count = 1
```

Next:

k

Create a new node.

Increment:

duck → count = 1

Current relevant counts:

d → 2
do → 1
dog → 1
du → 1
duc → 1
duck → 1

Insert **dove**

Start from root.

Character:

d

Existing node.

Increment:

d → count = 3

Next:

o

Existing node.

Increment:

```
do → count = 2
```

Next:

```
v
```

Create a new node.

Increment:

```
dov → count = 1
```

Next:

```
e
```

Create a new node.

Increment:

```
dove → count = 1
```

Final relevant counts:

```
d    → 3  
do   → 2  
dog  → 1  
du   → 1  
duc  → 1  
duck → 1  
dov  → 1  
dove → 1
```

16. Why `count == 1` Means Unique

Suppose we are processing:

duck

Walk through the Trie.

First:

d

Its count is:

3

Meaning:

dog
duck
dove

all use this prefix.

Therefore:

"d" is not unique.

Continue.

Next:

du

Its count is:

1

Only one word uses this prefix:

```
duck
```

Therefore:

```
"du" is unique.
```

Because we are walking from left to right, this is automatically the shortest unique prefix.

17. Optimized Algorithm

Step 1: Create a Trie

```
root = TrieNode()
```

Step 2: Insert Every Word

For every word:

```
Start at root.  
  
For every character:  
    Create the child if it does not exist.  
    Move to the child.  
    Increment the child's count.
```

Step 3: Find the Answer for Every Word

For every word:

```
Start at root.  
  
Create an empty prefix.  
  
For every character:  
    Move to the child.
```

Add the character to the prefix.

If node.count == 1:

return prefix

18. Optimized Code

```
class TrieNode:

    def __init__(self):

        self.children = [None] * 26

        self.count = 0

class Trie:

    def __init__(self):

        self.root = TrieNode()

    def insert(self, word):

        current = self.root

        for ch in word:

            index = ord(ch) - ord('a')

            if current.children[index] is None:

                current.children[index] = TrieNode()

            current = current.children[index]

            current.count += 1

    def get_unique_prefix(self, word):

        current = self.root

        prefix = ""

        for ch in word:
```



```
        index = ord(ch) - ord('a')

        current = current.children[index]

        prefix += ch

        if current.count == 1:

            return prefix

    return word

def shortest_unique_prefixes(arr):

    trie = Trie()

    # Step 1: Build the Trie
    for word in arr:

        trie.insert(word)

    # Step 2: Find the shortest unique prefix
    result = []

    for word in arr:

        result.append(trie.get_unique_prefix(word))

    return result
```

19. Complete Dry Run

Input:

```
arr = ["zebra", "dog", "duck", "dove"]
```

Step 1: Insert **zebra**

The path is:

```
z
ze
zeb
```

```
zebr  
zebra
```

Counts:

```
z      → 1  
ze     → 1  
zeb    → 1  
zebr   → 1  
zebra  → 1
```

Step 2: Insert **dog**

The path is:

```
d  
do  
dog
```

Counts:

```
d   → 1  
do  → 1  
dog → 1
```

Step 3: Insert **duck**

The **d** node already exists.

Update:

```
d → 2
```

Then create:

```
du  
duc  
duck
```

Counts:

```
d    → 2
du   → 1
duc  → 1
duck → 1
```

Step 4: Insert **dove**

The **d** node exists.

Update:

```
d → 3
```

The **do** node exists.

Update:

```
do → 2
```

Then create:

```
dov
dove
```

Counts:

```
d    → 3
do   → 2
dov  → 1
dove → 1
```

20. Final Trie Prefix Counts

Relevant prefixes:

Prefix	Count

z	1
d	3
do	2
dog	1
du	1
duc	1
duck	1
dov	1
dove	1

21. Query Phase Dry Run

Word: **zebra**

Walk:

z

Count:

1

Stop immediately.

Answer:

z

Word: **dog**

Walk:

d

Count:

3

Continue.

Walk:

do

Count:

2

Continue.

Walk:

dog

Count:

1

Stop.

Answer:

dog

Word: duck

Walk:

d

Count:

3

Continue.

Walk:

du

Count:

1

Stop.

Answer:

du

Word: **dove**

Walk:

d

Count:

3

Continue.

Walk:

do

Count:

2

Continue.

Walk:

dov

Count:

1

Stop.

Answer:

dov

22. Final Result

["z", "dog", "du", "dov"]

23. Dry Run of the Second Example

Input:

["geeksgeeks", "geeksquiz", "geeksforgeeks"]

The common prefix is:

geeks

Prefix counts:

```
g      → 3
ge     → 3
gee    → 3
geek   → 3
geeks  → 3
```

After that:

```
geeksgeeks
  |
  g

geeksquiz
  |
  q

geeksforgeeks
  |
  f
```

Therefore:

```
geeksg → count 1
geeksq → count 1
geeksf → count 1
```

Processing **geeksgeeks**

```
g      → 3
ge     → 3
gee    → 3
geek   → 3
geeks  → 3
geeksg → 1
```

Answer:

```
geeksg
```

Processing **geeksquiz**


```
g      → 3
ge     → 3
gee    → 3
geek   → 3
geeks  → 3
geeksq → 1
```

Answer:

```
geeksq
```

Processing **geeksforgeeks**

```
g      → 3
ge     → 3
gee    → 3
geek   → 3
geeks  → 3
geeksf → 1
```

Answer:

```
geeksf
```

24. Why the Trie Solution Is Better

Brute Force

For every word:

```
Try every prefix.
```

For every prefix:

```
Scan all words.
```

This repeats the same prefix comparisons many times.

Conceptually:

```
dog → check "d"  
duck → check "d" again  
dove → check "d" again
```

The same work is repeated.

Trie

The prefix is stored once.

For example:

```
"d"
```

is one Trie node.

Its count tells us immediately:

```
How many words use prefix "d"?
```

Similarly:

```
"do"
```

is one Trie node.

Its count tells us:

```
How many words use prefix "do"?
```

Therefore, instead of repeatedly comparing strings, we use the stored prefix information.

25. Complexity

Let:

N = number of words

L = maximum length of a word

Building the Trie

Each character of every word is processed once.

Therefore:

$$O(N \times L)$$

Finding the Answers

Each word is traversed once.

Therefore:

$$O(N \times L)$$

Total Time Complexity

$$O(N \times L)$$

Space Complexity

The Trie can contain at most:

$$O(N \times L)$$

unique character nodes.

With a fixed array of 26 children per node:

$$O(N \times L \times 26)$$

which is generally considered:

```
O(N × L)
```

because 26 is a constant.

26. Important Pattern

This problem follows the pattern:

```
Many strings
  ↓
Shared prefixes matter
  ↓
Build a Trie
  ↓
Store information at every prefix node
  ↓
Query each string
  ↓
Find the first node satisfying a condition
```

Here the condition is:

```
node.count == 1
```

Therefore:

```
First prefix where count == 1
=
Shortest unique prefix
```

27. General Trie Problem-Solving Framework

When you see a problem involving strings, ask:

Question 1

Do multiple strings share prefixes?

Examples:

```
apple
application
apply
```

If yes, think:

```
Trie
```

Question 2

What information should every Trie node store?

Possible information:

```
count
is_end
frequency
maximum value
minimum value
word index
number of words below
number of words passing through
```

For this problem:

```
children
count
```

Question 3

What condition am I looking for while walking through a word?

For this problem:

```
count == 1
```

Therefore:

```
First node where count == 1
=
Answer
```

28. Generic Trie Template

Node

```
class TrieNode:

    def __init__(self):

        self.children = [None] * 26

        self.count = 0
```

Insert

```
def insert(word):

    node = root

    for ch in word:

        index = ord(ch) - ord('a')

        if node.children[index] is None:

            node.children[index] = TrieNode()

        node = node.children[index]

    update information
```

Query

```
def query(word):

    node = root

    for ch in word:
```

```
index = ord(ch) - ord('a')

node = node.children[index]

check condition

if condition is satisfied:

    return answer
```

29. The Important Mental Model

Do not think:

"I need to compare every word with every other word."

Instead think:

"Every prefix is a path in the Trie."

Then:

"If I store how many words use each path,
I can immediately know when the path becomes unique."

This is the real reason the Trie works.

30. Interview Recognition Checklist

When you see words like:

```
prefix
common prefix
unique prefix
shortest prefix
starts with
autocomplete
dictionary
```

```
word search
lexicographical ordering
```

Ask:

```
Do many strings share prefixes?
```

If yes:

```
Think Trie.
```

Then ask:

```
What information should each Trie node store?
```

For this problem:

```
children
count
```

Then ask:

```
What condition gives the answer?
```

Here:

```
count == 1
```

Final logic:

```
Build Trie
↓
Count words passing through every node
↓
For every word, walk from left to right
↓
Stop at the first node where count == 1
```


↓

That prefix is the shortest unique prefix

31. One-Line Interview Explanation

I build a Trie while storing, at every node, the number of words passing through that prefix. Then for each word, I traverse its characters from left to right and return the first prefix whose Trie node has count 1, because that is the shortest prefix shared by exactly one word.

32. Final Revision Sheet

Problem

Find the shortest prefix that uniquely identifies every word.

Brute Force

```
For every word:  
  Try every prefix.  
  Count how many words start with it.  
  Stop when count == 1.
```

Complexity:

```
O(N2 × L2) approximately
```

Optimized

```
Build Trie.  
  
During insertion:  
  node.count += 1  
  
During lookup:  
  Move through the word.  
  
  First node where:  
    node.count == 1
```

is the answer.

Key Data Structure

Trie

Important Node Data

children
count

Key Condition

count == 1

Complexity

Time:

$O(N \times L)$

Space:

$O(N \times L)$

Final Pattern to Remember

Prefix Problem
↓
Shared Prefixes
↓
Trie

↓
Store Prefix Information
↓
Walk the Word
↓
First Prefix Satisfying Condition
↓
Answer

For this problem:

First prefix where count == 1
=
Shortest Unique Prefix